Internal AI Copilot

Portfolio report: structured extraction + light agent for support/ops tickets — free, end-to-end demo

Prepared by Shane Singh · November 2026
Contact: shaness220@gmail.com · github.com/shay2222/internal-ai-copilot

How to read this report. Every claim is tagged. **Demonstrated** means the work is in a public repository or was run and recorded, with a file or screenshot as evidence. **Stated** means it comes from employment history and is described in general terms. **Not yet done** means the capability is not claimed. The line between these categories is kept strict on purpose.

1. Executive summary

Support and operations teams receive unstructured tickets every day. Manual triage is slow and inconsistent. This project is a free, end-to-end demonstration of an internal AI copilot that turns raw ticket text into clean structured data: category, urgency, summary, and a recommended next action.

Demonstrated. I built a complete pipeline with extraction, schema validation, evaluation against labeled examples, and a simple interface. On the test set the system achieved **3/3 category accuracy** and **3/3 urgency accuracy**. Validation passed on all tested outputs. The full code, sample data, architecture notes, and evaluation results are public at github.com/shay2222/internal-ai-copilot.

The current extractor is deliberately rule-based so the entire demo runs for free and is fully reproducible. The design keeps extraction, validation, and the interface as separate layers so a real LLM structured-extraction call can replace the rules later without rewriting the rest of the system. The goal is to show the full loop that production AI teams care about: problem definition, structured output, validation, evaluation, and honest limits.

2. Purpose of this report

This report answers four questions for a reviewer evaluating applied AI / full-stack AI work:

1. Can I define a real internal workflow problem and ship an end-to-end solution?
2. Do I produce structured, validated output (not free-form text)?
3. Do I measure accuracy against labeled examples and report the numbers?
4. Do I separate concerns (extraction, validation, interface) and document limitations clearly?

3. Scope and method

Demonstrated.

Item	Choice
Problem	Unstructured support/ops tickets → structured triage data
Output schema	category, urgency, summary, recommended_next_action
Categories	billing · bug · feature_request · other
Urgency levels	low · medium · high
Extraction	Rule-based (fully free & reproducible); LLM-ready design
Validation	Schema checks on every field before returning output
Evaluation	Labeled test set; category & urgency accuracy reported
Interface	run_copilot() function + notebook demo
Hosting	Google Colab (development) + public GitHub repository

Not measured in this run (Not yet done): live LLM structured-extraction call, continuous learning from corrections, multi-user concurrent load, production authentication, or cost-per-ticket tracking under real traffic. Those are natural next steps once the core loop is proven.

4. Results

Demonstrated. Numbers and screenshots below come from the recorded Colab run. The same functions are in the public repository.

4.1 Summary of measured results

Metric	Result
Category accuracy	3 / 3

Urgency accuracy	3 / 3
Validation on test outputs	Passed (Valid: True)

4.2 Evidence from the Colab run

Extraction + validation on the first sample ticket (billing / high urgency). Validation returned True.

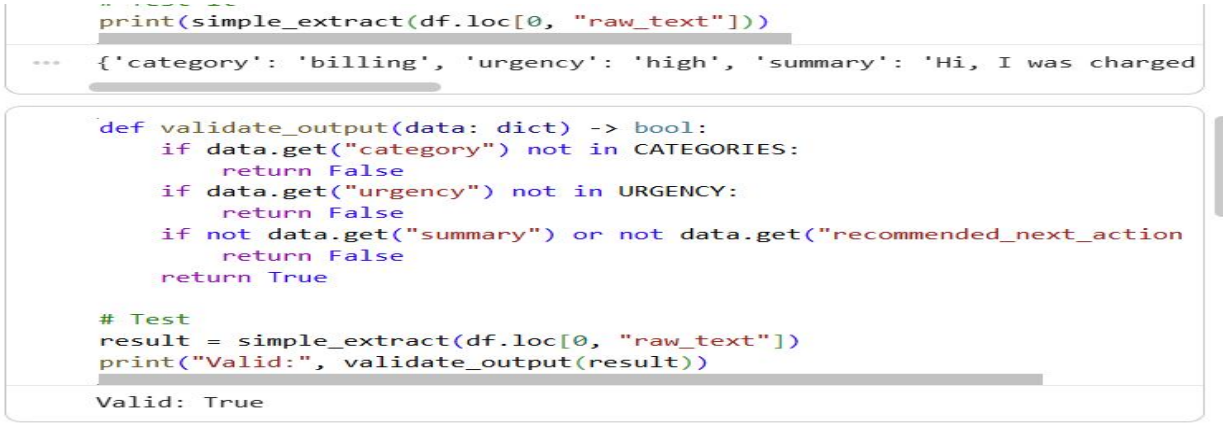


Figure 1 — simple_extract output and validate_output returning Valid: True.

Evaluation accuracy and a second successful run of the main interface function.

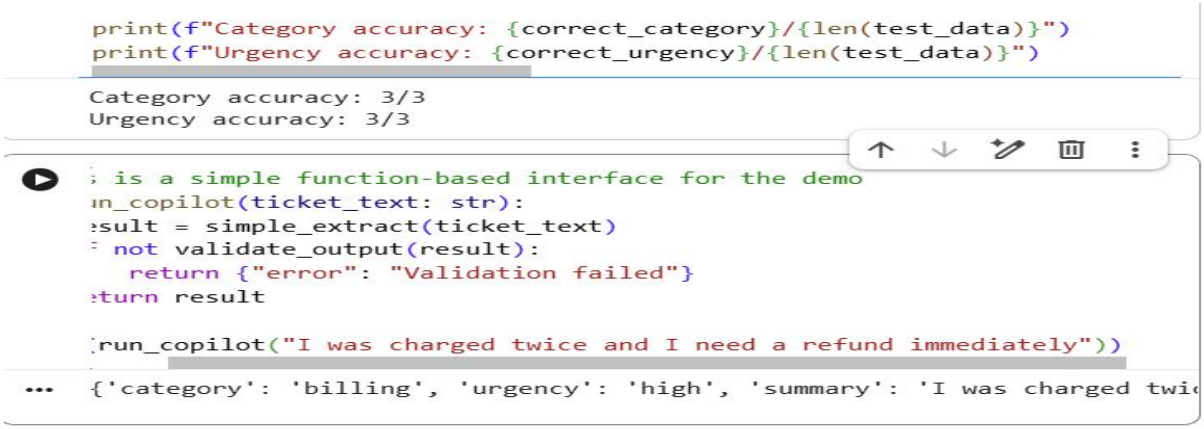


Figure 2 — Category accuracy 3/3, Urgency accuracy 3/3, and run_copilot() returning structured output.

5. Technical design

The system is intentionally split into small, replaceable pieces so each part can be improved independently.

Component	Responsibility
simple_extract()	Takes raw ticket text and returns a dict with category, urgency, summary, and recommended_next_action.
validate_output()	Checks that category and urgency belong to the allowed sets and that required fields are present. Returns False on any schema violation.
run_copilot()	Public interface. Calls extraction, runs validation, and either returns the structured result or an error dict.
Evaluation harness	Runs labeled examples, counts correct category and urgency predictions, and records the numbers.

The rule-based extractor exists only to keep the entire demo free and deterministic. The surrounding validation, interface, and evaluation layers are written so a real LLM structured-extraction call (for example using a JSON-mode or tool-calling model) can be dropped in later without changing how the rest of the system behaves.

6. What this shows

- End-to-end ownership of a narrow but realistic internal AI tool (problem → schema → extraction → validation → evaluation → interface).

• Discipline around structured output and schema validation — the same concerns that matter when shipping extraction features in production.
• Measurement against labeled examples instead of only showing a happy-path demo.
• Clear separation of layers and honest documentation of current limitations and next steps.
• Ability to ship a complete, public, reproducible artifact using only free tools (Colab + GitHub).

7. Limitations and next steps

Current limitations (stated clearly):

• Extraction is rule-based rather than model-based. This was a deliberate choice for free, fully reproducible results.
• Evaluation set is intentionally small (3 core test cases for the first version).
• No production authentication, rate limiting, or persistent logging yet.

Natural next steps:

• Replace simple_extract with an LLM structured-extraction call while keeping the same validation and evaluation harness.
• Expand the labeled evaluation set and add error analysis by category.
• Wrap the functions in a small FastAPI service and add basic request logging.
• Track simple cost and latency metrics once a model call is introduced.

8. Repository

All code, sample data, architecture notes, evaluation results, and this report's supporting material are public:

https://github.com/shay2222/internal-ai-copilot

Key paths: **src/copilot.py** (core functions), **docs/architecture.md**, **evaluation/results.md**, **notebooks/** (Colab demo), and the root README.
